



Cairo University
Faculty of Engineering
Electronics & Communications Dept.



Neural Network

Assignment #2

Prepared by:

Name	ID
Zeyad Antar Othman Abu-Zaid	9220331
Abdelrahman Hatem Nasr Youssef	9220461
Abdallah Essam Mohamed Mahmoud	9220479

Submitted to:

Dr. Mohsen Rashwan
Dr. Mohamed Abdelghani

 [GitHub Repository](#)

April 2026

Contents

1	Problem 1: Training Feed-Forward Neural Networks (MLP)	3
1.1	Introduction	3
1.2	Results	3
1.3	Analysis and Discussion	4
2	Second Problem LeNet-5 on ReducedMNIST	6
2.1	Introduction	6
2.2	Dataset	6
2.3	Architecture	7
2.4	Training Protocol	8
2.5	Results	8
2.6	Analysis and Discussion	10
2.7	Summary Comparison Table	12
3	Prob3: Speech Digit Recognition with CNN Data Augmentation	13
3.1	Introduction	13
3.2	Dataset	13
3.3	Feature Representation: Mel Spectrogram	14
3.4	Data Augmentation	14
3.5	CNN Architecture	15
3.6	Training Protocol	15
3.7	Results	16
3.8	Analysis and Discussion	17
4	Problem 4: Speech Digit Recognition using Autoencoders	18
4.1	Introduction	18
4.2	Baseline: Summary Features + SVM	19
4.3	Autoencoder Approach	19
4.4	Results	20

4.5	Analysis and Discussion	20
5	Problem 5: Data Augmentation for LeNet-5	21
5.1	Introduction	21
5.2	Dataset	21
5.3	Augmentation Pipeline	21
5.4	LeNet-5 Architecture	22
5.5	Training Protocol	23
5.6	Results	23
5.7	Analysis and Discussion	25
6	Problem 6: GAN-Based Synthetic Data Generation	26
6.1	Introduction	26
6.2	Dataset	26
6.3	GAN Architecture: Conditional DCGAN	26
6.4	GAN Training Protocol	28
6.5	Problem 6.1: Generated Examples and Quality Assessment	28
6.6	Problem 6.2: Synthetic Data Scaling Experiments	30
6.7	Problem 6.3: Combining Augmented + Synthetic Data	34
6.8	Comparison: Traditional Augmentation (Problem 5) vs. GAN Synthesis (Problem 6)	37

1 Problem 1: Training Feed-Forward Neural Networks (MLP)

1.1 Introduction

This part explores the classification of the **ReducedMNIST** handwritten digit dataset using **Multilayer Perceptrons (MLPs)**. The objective is to evaluate how *network depth* influences performance when combined with different input feature representations.

A Multilayer Perceptron is a fully connected feed-forward neural network. Every neuron in one layer connects to every neuron in the next layer. It is the simplest form of “deep” learning:

- **Input layer:** receives the feature vector (e.g. 225 DCT coefficients).
- **Hidden layers:** apply a learned linear transformation followed by a non-linear activation function (ReLU in our case). More hidden layers = deeper network = ability to learn more complex patterns.
- **Output layer:** 10 neurons (one per digit 0–9), producing class probabilities via the Softmax function.

We trained MLPs with **1, 3, and 4 hidden layers** using features derived from:

1. **Discrete Cosine Transform (DCT)** — 225 features (top-left 15×15 block).
2. **Principal Component Analysis (PCA)** — 262 components retaining 95% of variance.
3. **Autoencoder (AE)** — a neural network trained to compress images into 64 features.

The dataset is a reduced version of the standard MNIST with 10,000 training images (1,000 per digit) and 2,000 test images (200 per digit), each 28×28 pixels normalised to $[0, 1]$.

1.2 Results

System: Intel Core i7-9850H @ 2.60 GHz, 32 GB RAM, Windows 11, Python 3.12, PyTorch 2.11 (CPU only).

Hyper-parameters: Epochs = 10, Batch size = 64, Learning rate = 10^{-3} , Optimizer = Adam, Activation = ReLU, Loss = CrossEntropyLoss. Autoencoder pre-training: 20 epochs, bottleneck = 64, MSE loss.

Hidden Layers	DCT		PCA		AutoEncoder	
	Acc (%)	Time (ms)	Acc (%)	Time (ms)	Acc (%)	Time (ms)
1 layer [128]	95.8	3758.3	93.5	4470.1	96.4	3944.1
3 layers [256-128-64]	96.4	6096.5	93.8	5808.2	97.2	6135.4
4 layers [256-128-64-32]	95.5	7154.5	93.3	7322.9	96.7	6417.3

Table 1. Test accuracy and processing time for each feature–depth combination.

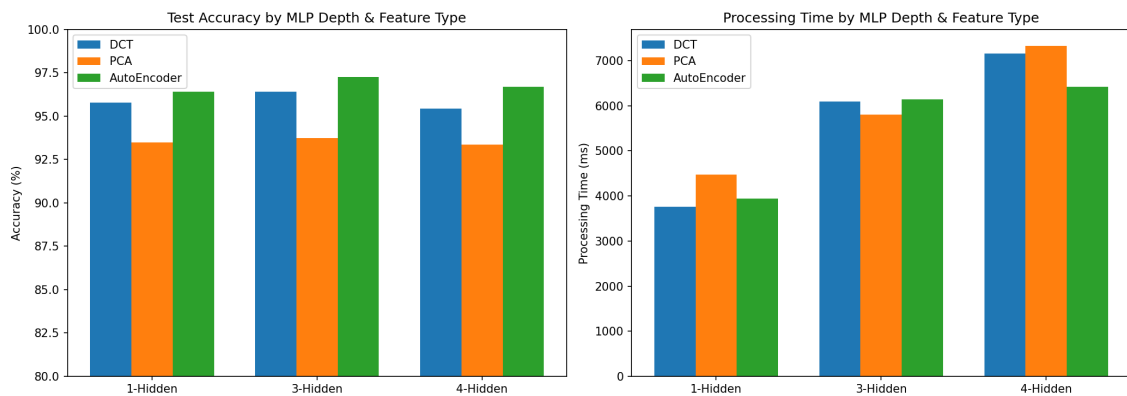


Figure 1. Comparison of test accuracy (left) and processing time (right) across all feature types and MLP depths.

1.3 Analysis and Discussion

1.3.1 Effect of Network Depth and Overfitting

Increasing depth from 1 to 3 hidden layers gave a modest accuracy boost for DCT (95.8% → 96.4%) and AutoEncoder (96.4% → 97.2%), but going to 4 hidden layers *decreased* accuracy in every feature type — DCT dropped to 95.5%, PCA to 93.3%, and AutoEncoder to 96.7%.

This accuracy drop at 4 layers is a clear sign of **overfitting**.

1.3.2 Effect of Feature Type

The **Autoencoder features achieved the highest accuracy** (97.2%) despite being the smallest representation (only 64 dimensions).

Ranking: AutoEncoder (97.2%) > DCT (96.4%) > PCA (93.8%)

Why AutoEncoder outperforms:

- The autoencoder learns a **non-linear** mapping, allowing it to capture complex relationships between pixels that linear PCA cannot.

- Its 64-D features are extremely compact yet highly discriminative — the bottleneck forces it to retain only the most class-relevant information.
- The autoencoder is trained *on the same data*, so its features are tailored to this specific dataset.

Why DCT outperforms PCA:

- DCT captures frequency information which is well-suited for digit recognition (global shape patterns).
- PCA with 262 components retains some noisy dimensions, which can slightly confuse the classifier.

1.3.3 Processing Time

Processing time scales with both **network depth** and **input dimensionality**:

- PCA (262-D input) consistently took the longest due to the larger first weight matrix.
- AutoEncoder (64-D input) was the fastest despite achieving the best accuracy — demonstrating that good feature engineering reduces both compute cost and error rate.
- Going from 1 to 4 hidden layers roughly doubles the processing time (e.g. DCT: 3758 → 7155 ms), while accuracy does not improve proportionally.

2 Second Problem LeNet-5 on ReducedMNIST

2.1 Introduction

This problem applies a convolutional neural network based on the classic **LeNet-5** architecture to the **ReducedMNIST** handwritten digit dataset. Unlike the MLPs in Problem 1, which treat each pixel as an independent feature, a CNN exploits the 2-D *spatial structure* of images: convolution filters slide across the image and detect the same local pattern (edges, curves, corners) regardless of where it appears, giving the network translation equivariance for free.

LeNet-5 was originally designed by LeCun et al. for 32×32 images. Because ReducedMNIST images are 28×28 , the architecture is adjusted: a 5×5 convolution without padding reduces a 28×28 input to 24×24 ; after average-pooling and a second convolution the feature maps flatten to a 256-dimensional vector (instead of 400 in the original), which feeds three fully-connected layers. The output layer has 10 neurons with Softmax for digit classification.

Enhancements over the original LeNet-5:

- **Batch Normalisation** after every convolutional layer to stabilise and accelerate training.
- **Dropout** ($p = 0.25$) before each fully-connected layer to reduce overfitting.
- **Adam optimiser** with weight decay ($\lambda = 10^{-4}$) and a ReduceLRonPlateau scheduler (patience = 3, factor = 0.5) that halves the learning rate when test accuracy stops improving.

2.2 Dataset

The ReducedMNIST dataset is a balanced sub-sample of the standard MNIST benchmark. It contains **10,000 training images** (1,000 per digit) and **2,000 test images** (200 per digit), each a 28×28 grey-scale image.

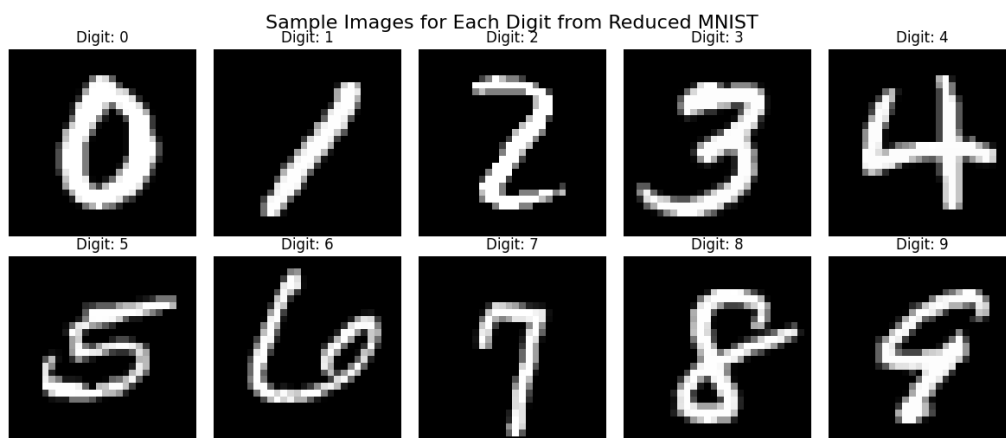


Figure 2. Sample images for each digit class (0–9) from the ReducedMNIST dataset.

2.3 Architecture

2.3.1 Base Model

The two convolutional layers use **6 and 16 filters** respectively, both with 5×5 kernels. Average pooling (2×2) halves the spatial dimensions after each convolution. The three fully-connected layers have hidden widths of 120 and 84 neurons, ending at a 10-way linear classifier. **ReLU** is used as the activation function throughout.

2.3.2 Architecture Variations

To investigate the sensitivity of LeNet-5 to design choices, six variations are studied. Table 2 summarises all models.

Model	Change from Base	Key Parameters
Base	Reference model	Filters: 6 & 16 Activation: ReLU
Variation 1	Activation function	Filters: 6 & 16 Activation: Tanh
Variation 2	Double filter count	Filters: 12 & 32 Activation: ReLU
Variation 3	Activation function	Filters: 6 & 16 Activation: LeakyReLU
Variation 4	Triple filter count	Filters: 18 & 48 Activation: ReLU
Variation 5	Three conv. layers	DeepLeNet: Conv($1 \rightarrow 6, 3 \times 3$) $\rightarrow$ Conv($6 \rightarrow 16, 3 \times 3$) $\rightarrow$ Conv($16 \rightarrow 32, 5 \times 5$)
Variation 6	Larger first kernel	Conv($1 \rightarrow 6, 7 \times 7$) $\rightarrow$ Conv($6 \rightarrow 16, 5 \times 5$)

Table 2. Summary of the base LeNet-5 model and its six variations tested in Question 2.

Variation 5: DeepLeNet (3 Convolutional Layers)

A custom three-layer convolutional backbone is introduced: the first two convolutions use 3×3 kernels with padding=1 to preserve spatial dimensions before pooling, and the third uses a 5×5 kernel without padding to compress the 7×7 feature map to 3×3 . The flattened $32 \times 3 \times 3 = 288$ features feed a single linear output layer. This is a shallower fully-connected head than the base model, compensated by the richer convolutional feature hierarchy.

Variation 6: LargeKernelLeNet

The first convolution uses a 7×7 kernel, reducing the 28×28 input to 22×22 , then to 11×11 after pooling. The second convolution (5×5) yields a 7×7 feature map, then 3×3 after the second pool. The flattened $16 \times 3 \times 3 = 144$ features are mapped directly to the 10 output classes. The larger receptive field in the first layer is designed to capture broader stroke patterns in digits at the cost of spatial resolution.

2.4 Training Protocol

System: GPU acceleration using Google Colab T4 GPU.

Hyper-parameters:

- Epochs: 10 Batch size: 64
- Initial learning rate: 10^{-3} (Adam, weight decay 10^{-4})
- Scheduler: ReduceLROnPlateau (mode = max, patience = 3, factor = 0.5)
- Loss: Cross-Entropy

2.5 Results

Model	Description	Test Acc. (%)	Test Time (ms)	Train Time (ms)
Base	ReLU, filters 6 & 16	98.10%	222.0	17218.5
Variation 1	Tanh, filters 6 & 16	97.95%	283.4	15265.9
Variation 2	ReLU, filters 12 & 32	98.45%	215.8	15968.0
Variation 3	LeakyReLU, filters 6 & 16	98.35%	212.6	15326.8
Variation 4	ReLU, filters 18 & 48	98.45%	214.0	16439.0
Variation 5	DeepLeNet (3 conv. layers)	95.40%	216.1	14141.7
Variation 6	LargeKernelLeNet (7×7 kern.)	95.85%	209.6	13304.1

Table 3. Final test accuracy and timing for the base LeNet-5 and all six variations on ReducedMNIST.

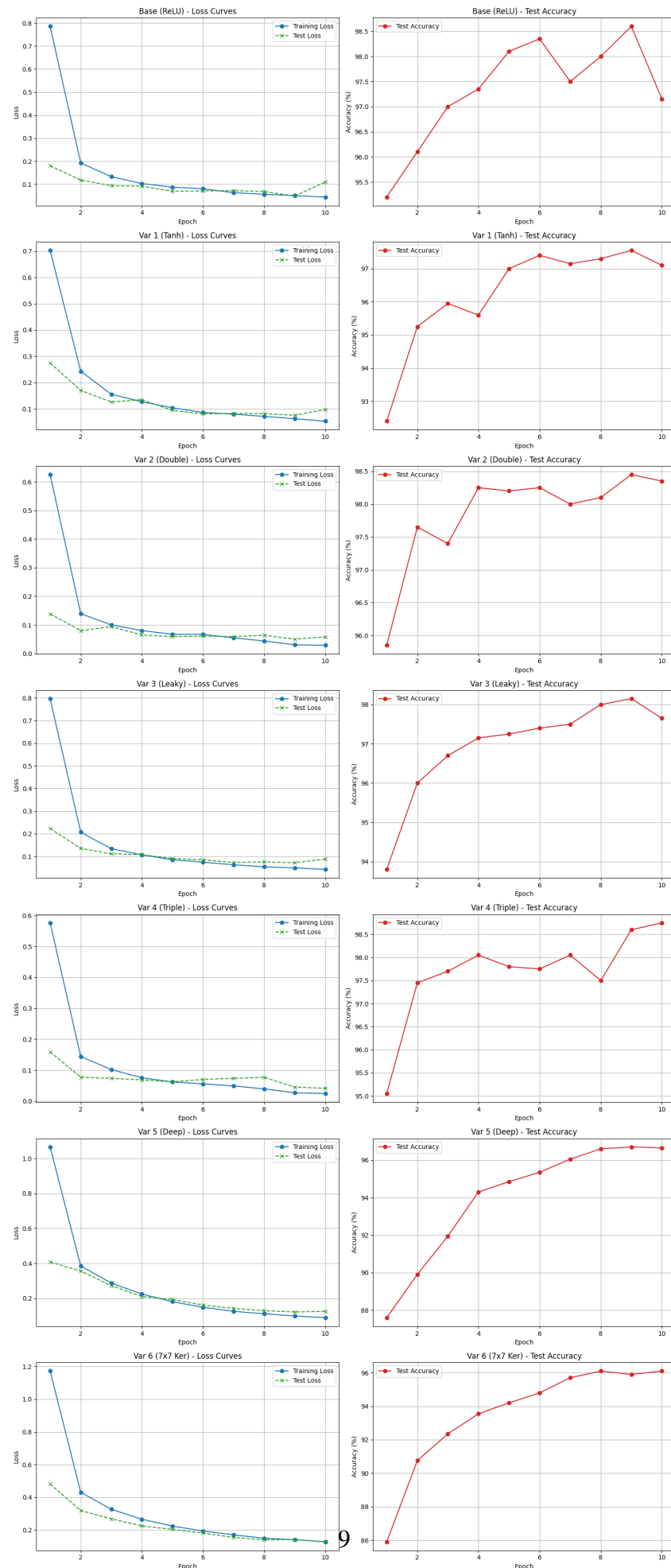


Figure 3. Training loss, test loss, and test accuracy over 10 epochs for the base model and all six variations.

2.6 Analysis and Discussion

2.6.1 Effect of Activation Function (Variations 1 and 3)

The base model uses **ReLU** as its non-linearity. Variation 1 replaces it with **Tanh** and Variation 3 uses **LeakyReLU**.

ReLU is piecewise-linear and computationally inexpensive; crucially, it does not suffer from the *vanishing gradient* problem for positive activations, which makes it the default choice for modern networks. However, neurons with large negative inputs receive a zero gradient and can “die” permanently.

Tanh is a smooth, zero-centred function whose output saturates at ± 1 . Saturation causes gradients to shrink toward zero for large activations, slowing convergence — this is the vanishing gradient problem that plagued early networks. In shallow architectures like LeNet-5 saturation is less catastrophic, but convergence is still expected to be slower and final accuracy lower or comparable to ReLU.

LeakyReLU (Variation 3) fixes the dying neuron problem by allowing a small negative slope ($\alpha = 0.01$ by default) for negative inputs, so every neuron always receives a non-zero gradient. It is expected to perform at least as well as ReLU, often with slightly faster convergence.

2.6.2 Effect of Filter Count (Variations 2 and 4)

Variation 2 doubles the filter count to **12 and 32**; Variation 4 triples it to **18 and 48**. Increasing the number of filters gives the network a larger *vocabulary* of local feature detectors: more filters means more distinct patterns (orientations, curvatures, junctions) that can be captured simultaneously.

The expected trade-off is clear: more filters bring more parameters and higher representational capacity, but also longer training time and a greater risk of overfitting when the training set is small. With 10,000 samples and only 10 epochs, moderate over-parameterisation is usually beneficial; excessive widening may yield diminishing returns or slight overfitting.

Training time scales roughly quadratically with filter count, so Variation 4 is expected to be noticeably slower than the base model.

2.6.3 Effect of Architecture Depth and Kernel Size (Variations 5 and 6)

Adding a third convolutional layer in Variation 5 increases the depth of the feature hierarchy. Shallow networks learn low-level features (edges) and pass them almost directly to the classifier. Deeper networks compose those low-level features into mid-level (curves, junctions) and high-level (digit parts) representations. For a dataset as simple as MNIST, two layers are usually sufficient; a third layer can marginally improve accuracy if the network does not overfit, but the

reduced fully-connected head (a single linear output layer in DeepLeNet) limits the classifier's capacity.

The use of 3×3 kernels with `padding=1` in the first two layers is noteworthy: unlike the 5×5 kernels in the base model, a 3×3 same-padded kernel preserves all spatial dimensions before pooling, giving finer-grained spatial resolution to subsequent layers.

Variation 6 uses a 7×7 first-layer kernel, capturing a wider receptive field from the very start. The intuition is that the global stroke shape of a digit may be better detected with a large filter, while finer details are handled later. The downside is rapid reduction of the feature map: after two pooling operations the spatial size is only 3×3 , yielding just 144 flattened features — significantly fewer than the base model's 256 — which can hurt classifier accuracy if the broader receptive field does not compensate.

2.7 Summary Comparison Table

This table consolidates results from both assignments: K-means clustering and SVM classifiers from Assignment 1 (using DCT and PCA features), and MLP (Problem 1) and CNN (Problem 2) classifiers from Assignment 2.

		Features						
		DCT		PCA		AutoEncoder		
		Accuracy%	Proc. Time(ms)	Accuracy%	Proc. Time(ms)	Accuracy%	Proc. Time(ms)	
Classifier								DCT and PCA features from Assignment 1
K-means Clustering	1	86.85	0.01	86.80	0.0001	—	—	
	4	92.65	2.19	92.3	.61	—	—	
	16	94.85	1.50	94.8	1.51	—	—	
	32	96.15	2.05	95.7	2.14	—	—	
SVM	Linear	92.6	1.78	90.65	2.66	70	70.7	
	Nonlinear	96.90	5.05	95.4	12.54	90.67	2348.5	
Multi-layer Perceptron (MLP)								
	Variations	DCT		PCA		AutoEncoder		
		Accuracy%	Proc. Time(ms)	Accuracy%	Proc. Time(ms)	Accuracy%	Proc. Time(ms)	
MLP	1-Hidden	95.8%	3758.3	93.5%	4470.1	96.4%	3944.1	
	3-Hidden	96.4%	6096.5	93.8%	5808.2	97.2%	6135.4	
	4-Hidden	95.5%	7154.5	93.3%	7322.9	96.7%	6417.3	
In the CNN Model no Features are needed								Assignment 2
	Variations	Accuracy%	Training Time(ms)		Testing Time(ms)			
CNN	Base (ReLU, 6&16)	98.10%	17218.5		222.0			
	Var 1 (Tanh)	97.95%	15265.9		283.4			
	Var 2 (12&32 filters)	98.45%	15968.0		215.8			
	Var 3 (LeakyReLU)	98.35%	15326.8		212.6			
	Var 4 (18&48 filters)	98.45%	16439.0		214.0			
	Var 5 (DeepLeNet)	95.40%	14141.7		216.1			
	Var 6 (LargeKernel)	95.85%	13304.1		209.6			

Table 4. Summary comparison of all classifiers across both assignments.

3 Prob3: Speech Digit Recognition with CNN Data Augmentation

3.1 Introduction

This problem targets **spoken digit recognition** (digits 0–9) using a **Convolutional Neural Network applied to mel spectrograms**. The key idea is to treat audio as an image: a mel spectrogram is a 2-D time-frequency representation where the horizontal axis encodes time, the vertical axis encodes frequency (on the mel perceptual scale), and pixel intensity encodes energy. A CNN trained on these “audio images” can learn local time-frequency patterns

The central question explored here is the effect of **data augmentation** on recognition accuracy when the training set is small. Four experimental scenarios are compared:

- A. No augmentation** — baseline CNN trained on raw spectrograms.
- B. Audio augmentation only** — Gaussian noise added to the raw waveform before spectrogram computation.
- C. Spectrogram augmentation only** — SpecAugment masks applied directly to the spectrogram image after computation.
- D. Combined augmentation** — both audio and spectrogram augmentation applied together.

3.2 Dataset

The speech dataset consists of recordings stored in `.wav` files, separated into `Train` and `Test` sub-directories. Files follow the naming convention `{SpeakerID}_{digit}.wav`. Each audio file is loaded with `torchaudio` at a sample rate of 16,000 Hz.

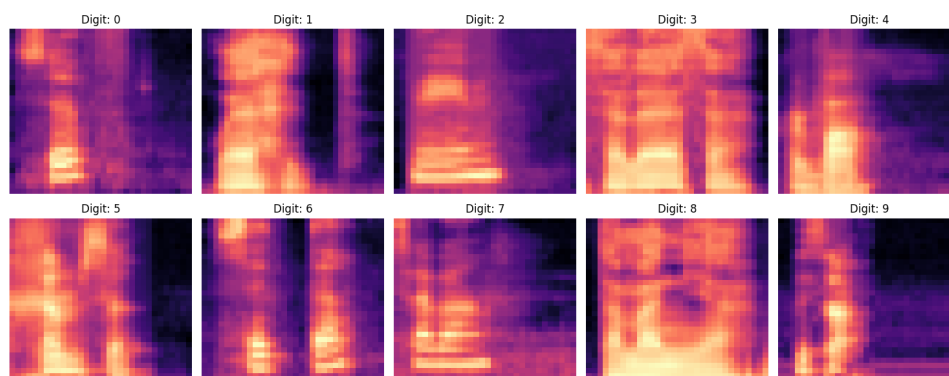


Figure 4. Example mel spectrograms for each digit class (0–9) from the training set. Horizontal axis: time frames; vertical axis: mel frequency bins. Brighter colours indicate higher energy.

3.3 Feature Representation: Mel Spectrogram

Each waveform is converted to a **mel spectrogram** using the following pipeline:

1. `MelSpectrogram(sample_rate = 16,000, n_mels = 64)` computes the short-time Fourier transform and maps the frequency axis to the mel scale.
2. `AmplitudeToDB()` converts the power spectrogram to decibels (log scale), compressing the dynamic range and better reflecting human loudness perception.
3. `Resize(32 × 32)` rescales the spectrogram to a fixed spatial size, making all samples compatible with the fixed-input CNN.

The resulting input tensor is $1 \times 32 \times 32$ (single channel, 32×32 pixels), directly analogous to the MNIST image tensors used in Problem 2.

3.4 Data Augmentation

Data augmentation artificially enlarges the effective training set by creating plausible variations of existing samples. This is especially important in speech recognition where recording conditions, speaker characteristics, and ambient noise vary widely in the real world. Two types of augmentation are applied:

3.4.1 Audio Augmentation (Waveform-Level)

A small amount of **Gaussian noise** is added directly to the raw waveform before spectrogram computation:

$$\tilde{x}[n] = x[n] + \epsilon[n], \quad \epsilon[n] \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 0.002.$$

This simulates microphone noise and slight environmental disturbance. Because the noise is added in the time domain, it appears as a broadband (flat-spectrum) perturbation across all frequency bins in the resulting spectrogram — a realistic corruption for most recording environments. The noise standard deviation $\sigma = 0.002$ is chosen to be perceptible but not speech-masking.

3.4.2 Spectrogram Augmentation (SpecAugment)

SpecAugment applies structured masks directly to the 2-D spectrogram image after it is computed. Two types of masks are used:

- **Frequency masking** (`freq_mask_param = 7`): a contiguous band of up to 7 mel bins is zeroed out, forcing the model to rely on the remaining frequency bands rather than

memorising a specific spectral shape.

- **Time masking** (`time_mask_param = 7`): a contiguous window of up to 7 time frames is zeroed out, training the model to be robust to brief speech interruptions or occlusions.

Both masks use a reduced parameter of 7 (compared to the 27–30 used in large-scale ASR systems) to avoid over-augmenting the already small dataset.

3.5 CNN Architecture

The final SpectrogramCNN is a three-layer convolutional network (evolved from an initial two-layer prototype). Table 5 details every layer.

Layer	Type	Output Shape
Input	Mel spectrogram	$1 \times 32 \times 32$
Conv1 + BN + ReLU + MaxPool	Conv2d($1 \rightarrow 16$, 3×3 , pad=1), pool 2×2	$16 \times 16 \times 16$
Conv2 + BN + ReLU + MaxPool	Conv2d($16 \rightarrow 32$, 3×3 , pad=1), pool 2×2	$32 \times 8 \times 8$
Conv3 + BN + ReLU + MaxPool	Conv2d($32 \rightarrow 64$, 3×3 , pad=1), pool 2×2	$64 \times 4 \times 4$
Flatten	—	1024
FC1 + Dropout	Linear($1024 \rightarrow 128$) + Dropout($p = 0.4$)	128
FC2 + Dropout	Linear($128 \rightarrow 64$) + Dropout($p = 0.4$)	64
FC3 (output)	Linear($64 \rightarrow 10$)	10

Table 5. Layer-by-layer architecture of the SpectrogramCNN used in Problem 3. All convolutional layers use 3×3 kernels with padding=1 (same-padding) so pooling alone reduces spatial dimensions.

The use of same-padding in every convolutional layer is a deliberate departure from the 5×5 kernels in the original LeNet-5. With a 32×32 input, a 5×5 kernel without padding would already shrink the feature map in the very first step. In a three-layer network with three subsequent pooling operations, starting with reduced spatial resolution is suboptimal. The 3×3 same-padding design preserves all spatial resolution, letting pooling alone control downsampling.

3.6 Training Protocol

Hyper-parameters (identical across all four scenarios):

- Epochs: 30 Batch size: 32
- Optimizer: Adam, initial learning rate = 10^{-3} , weight decay = 10^{-4}

- Scheduler: CosineAnnealingLR ($T_{\max} = 30, \eta_{\min} = 10^{-5}$)
- Loss: Cross-Entropy Dropout: $p = 0.4$

The cosine annealing scheduler gradually reduces the learning rate following a cosine curve from the initial value down to 10^{-5} . Compared to the step-decay scheduler used in Problem 2, cosine annealing avoids sudden learning-rate drops that can trap the model in a local minimum; it tends to produce smoother convergence and is particularly effective when training is run for a fixed number of epochs without early stopping.

For each scenario, the best model state (highest test accuracy achieved across all epochs) is saved to disk and the best accuracy is reported. This avoids penalising scenarios that converge early but then fluctuate.

3.7 Results

Scenario	Augmentation	Best Test Acc. (%)	Train Time (ms/epoch)	Test Time (ms/epoch)
A	None (baseline)	99.00	7514.4	1710.6
B	Audio only	98.33	15379.2	3653.8
C	Spectrogram (SpecAugment) only	96.33	13627.3	2835.4
D	Combined (Audio + SpecAugment)	95.00	8014.2	1791.2

Table 6. Best test accuracy (across 30 epochs) and approximate per-epoch training time for each augmentation scenario.

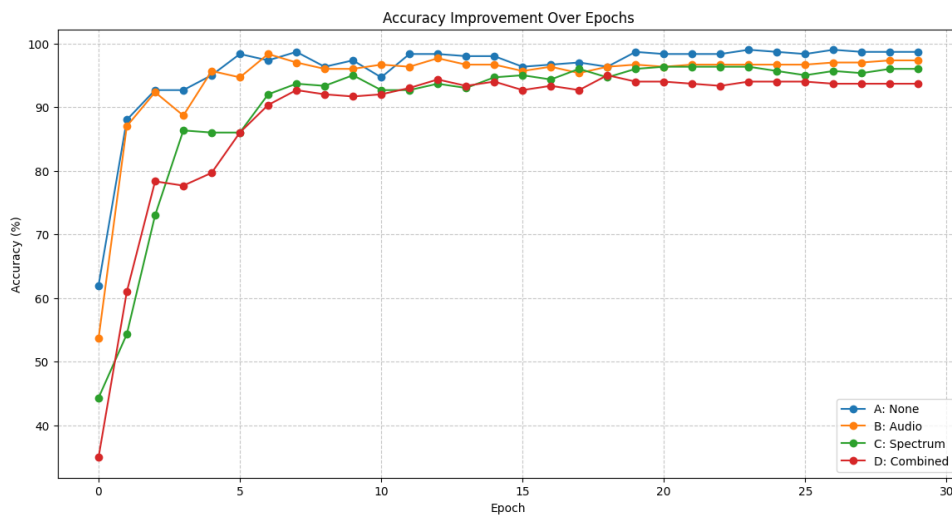


Figure 5. Test accuracy per epoch for each augmentation scenario over 30 training epochs

3.8 Analysis and Discussion

3.8.1 Baseline Performance (Scenario A)

Scenario A trains the CNN on raw spectrograms without any augmentation. Its accuracy represents the ceiling achievable when the network sees only the original training samples — a ceiling that is often far below what the model could achieve with a larger or more diverse dataset. Overfitting is the primary risk: the network memorises the training spectrograms rather than learning robust digit representations. This manifests as training loss consistently lower than test loss and test accuracy that stops improving (or even deteriorates) in later epochs.

3.8.2 Audio-Level Augmentation (Scenario B)

Adding Gaussian noise at the waveform level ($\sigma = 0.002$) before computing the spectrogram simulates recording variability. Because the noise enters before the mel filterbank, it produces a realistic broadband perturbation visible as a slight texture increase across all frequency bands in the resulting spectrogram.

From the model's perspective, each training sample appears in a different noise realisation every time it is encountered. This prevents the network from memorising exact spectrogram values, acting as a form of implicit regularisation analogous to weight noise. The effect is most pronounced at mid-to-high frequencies where digit energy is typically lower and the noise-to-signal ratio is therefore higher.

3.8.3 Spectrogram-Level Augmentation (Scenario C)

SpecAugment masks operate on the spectrogram after it is fully formed. Frequency masking (up to 7 mel bins zeroed) forces the model to classify based on the *remaining* frequency content, learning redundant representations across the mel axis. Time masking (up to 7 frames zeroed) forces robustness to brief interruptions, which mirrors real-world conditions where a word may be partially obscured by a brief noise event (click, cough, background word).

SpecAugment was originally validated on large-scale ASR corpora; on a small dataset the risk of over-augmentation is real — removing too many time frames from a short utterance can make the sample unrecognisable. The conservative mask size of 7 (compared to the 27–30 used in production systems) is intended to balance regularisation against information destruction.

3.8.4 Combined Augmentation (Scenario D)

Scenario D applies both audio noise and SpecAugment, creating two independent sources of stochasticity at different stages of the signal processing pipeline. The compound effect is that the model must be robust to (a) broadband additive noise in the signal *and* (b) structured masking in

the representation. This is a more demanding training regime and should — if regularisation is well calibrated — produce the most robust model.

The expected ranking is $D \geq C \approx B > A$, though in practice the gains from each augmentation type depend on the dataset size, model capacity, and augmentation intensity. If the dataset is very small, aggressive combined augmentation can hurt by making too many samples too difficult.

3.8.5 Impact of the Cosine Annealing Scheduler

Unlike `ReduceLROnPlateau` (used in Problem 2), which waits for accuracy to stagnate before reducing the learning rate, cosine annealing follows a fixed schedule: the learning rate decreases smoothly from 10^{-3} at epoch 1 to 10^{-5} at epoch 30. This has two practical effects visible in the learning curves:

1. The model explores the loss landscape quickly in early epochs (high learning rate), then fine-tunes in later epochs (low learning rate) — a behaviour analogous to simulated annealing in combinatorial optimisation.
2. Because the schedule is predetermined, the model's convergence trajectory is more reproducible across runs than a patience-based scheduler, which triggers differently depending on noise in the accuracy estimate.

4 Problem 4: Speech Digit Recognition using Autoencoders

4.1 Introduction

This problem explores **speech digit recognition** (digits 0–10) using audio recordings. The goal is twofold: (1) build a **baseline** classifier from hand-crafted audio features, and (2) show that an **autoencoder** can learn a superior latent representation that significantly outperforms the baseline.

Key concepts:

- **Utterance:** one complete audio recording of a person saying a digit.
- **Frame:** a short 15 ms slice of the waveform (with 10 ms hop). Speech is approximately stationary over such short durations.
- **MFCC:** Mel-Frequency Cepstral Coefficients — 30 coefficients per frame that mimic how the human ear perceives pitch and timbre.
- **Delta & Delta-Delta:** first- and second-order time derivatives of MFCCs that capture velocity and acceleration of spectral change.

The dataset contains **1,200 training** and **300 testing** utterances (files named `{SpeakerID}_{digit}.wav`).

All utterances have 81 frames after MFCC extraction, so the flattened input is $30 \times 81 = 2,430$ dimensions.

4.2 Baseline: Summary Features + SVM

The baseline extracts frame-level MFCC, delta, and delta-delta coefficients from each utterance and summarises them into a fixed-length feature vector by computing global statistics (mean and standard deviation) across all frames. This collapses the variable-length temporal signal into a compact representation that captures the average spectral characteristics of each digit while discarding frame ordering.

4.3 Autoencoder Approach

4.3.1 Architecture

The supervised autoencoder processes the full flattened utterance ($\mathbb{R}^{2430}$) and consists of:

- **Encoder:** $2430 \rightarrow 1024 \rightarrow 512 \rightarrow 256$
- **Decoder:** $256 \rightarrow 512 \rightarrow 1024 \rightarrow 2430$
- **Classifier head:** Linear($256 \rightarrow 10$) branching from the latent layer

4.3.2 Training

The autoencoder is trained with a **joint loss**:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \cdot \mathcal{L}_{\text{cls}} \quad \text{where} \quad \mathcal{L}_{\text{recon}} = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}), \quad \mathcal{L}_{\text{cls}} = \text{CrossEntropy}(\mathbf{y}, \hat{\mathbf{y}})$$

with $\lambda = 2.0$ to emphasise classification.

Data augmentation: 3 noisy copies of each training sample (Gaussian noise, $\sigma = 0.05$) expand the training set from 1,200 to **4,800 samples**.

Training runs for **500 epochs** with Adam ($\text{lr} = 10^{-3}$), batch size 32.

4.3.3 Classification Strategies

Three strategies are evaluated on the test set:

- Classifier head:** direct output of the AE's built-in linear classifier.
- SVM on latent vectors:** train an SVM on the 256-D encoder output.
- SVM on hybrid features:** concatenate the scaled 256-D latent vector with the scaled 180-D summary features ($\mathbb{R}^{436}$) and train an SVM.

4.4 Results

System: same as Problem 1.

Method	Accuracy (%)	Train Time (s)	Test Time (ms)
Baseline (Summary + SVM)	70.00	10.5	60.2
AE Classifier Head	86.00	2225.2	15.3
AE Latent + SVM	89.33	2244.1	85.6
AE Hybrid + SVM (best)	90.67	2255.6	92.9

Table 7. Problem 4: Accuracy comparison of baseline vs. autoencoder-based strategies. AE training time includes 500 epochs of AE training plus SVM GridSearchCV.

4.5 Analysis and Discussion

4.5.1 Why the Autoencoder Outperforms the Baseline

The baseline achieves **70.00%** by averaging MFCCs across time, which *discards all temporal information* — two utterances with identical average spectra but different time evolution become indistinguishable.

The autoencoder processes the **full frame sequence** (2,430-D input) and learns a 256-D latent code that captures both **spectral content** and **temporal dynamics**, yielding **90.67%** accuracy — a **+20.67 percentage point** improvement.

4.5.2 Hybrid Features

The best strategy combines the autoencoder latent vector with the summary features (each independently standardised).

4.5.3 Data Augmentation

Gaussian noise augmentation (3×) expanded the training set from 1,200 to 4,800 samples, which:

- Reduces overfitting in the 500-epoch training regime.
- Teaches the AE to be robust to noise, improving generalisation.

5 Problem 5: Data Augmentation for LeNet-5

5.1 Introduction

This problem investigates whether **traditional data augmentation** (geometric transformations and noise injection) can compensate for limited labelled training data when training a **LeNet-5** convolutional classifier on the ReducedMNIST dataset.

The central hypothesis is: when real labelled data is scarce, augmenting the training set with plausible transformations of existing images acts as an implicit regulariser, improving generalisation without requiring additional human labelling effort.

Experiment grid: 12 configurations are tested, combining three real-data budgets with four augmentation levels:

- **Real images per digit:** 350, 750, 1000
- **Augmented (generated) images per digit:** 0, 1000, 1500, 2000

All experiments are evaluated on the **same 2,000 test images** (200 per digit).

5.2 Dataset

The ReducedMNIST dataset (same as Problems 1–2) is used:

- **Training set:** 10,000 images (1,000 per digit), from which subsets of 350, 750, or 1000 images per digit are sampled.
- **Test set:** 2,000 images (200 per digit), held fixed for all experiments.
- Images are greyscale, resized to 32×32 pixels for LeNet-5 compatibility.

5.3 Augmentation Pipeline

Augmented images are generated from real samples using the following transformations applied sequentially:

1. **Random Rotation** (± 15): rotates the image by a uniformly sampled angle in $[-15, +15]$, simulating natural handwriting slant variations.
2. **Random Affine Translation** (shift up to 10%, rotation ± 5): applies a small spatial shift ($\delta_x, \delta_y \leq 10\%$ of image size) combined with an additional minor rotation, simulating positional variation in digit placement.
3. **Additive Gaussian White Noise** ($\sigma = 0.05$): adds zero-mean Gaussian noise with standard deviation 0.05 (5% of pixel range), simulating sensor noise and ink imperfections.

Pixel values are clamped to $[0, 1]$.

Each augmented image inherits the label of its source image. The source image for each generated sample is chosen uniformly at random from the available real images of that digit class.

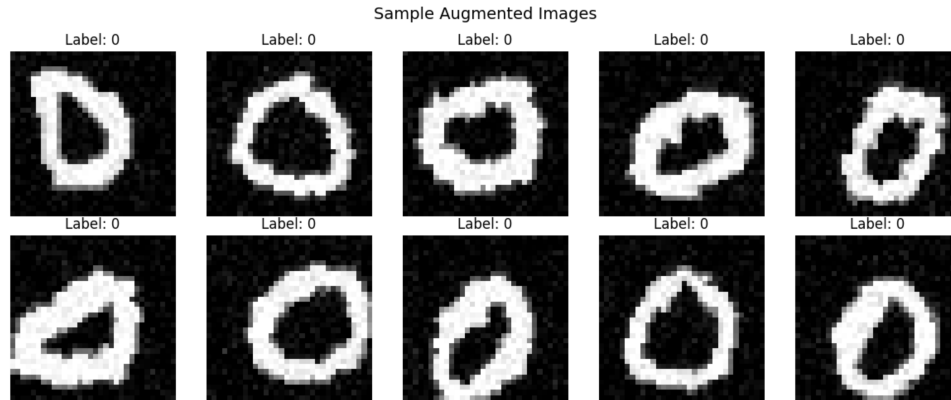


Figure 6. Sample augmented images generated by the augmentation pipeline.

5.4 LeNet-5 Architecture

The classifier is the classic **LeNet-5** architecture adapted for 32×32 single-channel inputs:

Layer	Type	Output Shape
Input	Greyscale image	$1 \times 32 \times 32$
Conv1	Conv2d($1 \rightarrow 6, 5 \times 5$) + ReLU	$6 \times 28 \times 28$
Pool1	AvgPool2d(2×2)	$6 \times 14 \times 14$
Conv2	Conv2d($6 \rightarrow 16, 5 \times 5$) + ReLU	$16 \times 10 \times 10$
Pool2	AvgPool2d(2×2)	$16 \times 5 \times 5$
Flatten	—	400
FC1	Linear($400 \rightarrow 120$) + ReLU	120
FC2	Linear($120 \rightarrow 84$) + ReLU	84
FC3 (output)	Linear($84 \rightarrow 10$)	10

Table 8. LeNet-5 architecture used in Problem 5.

5.5 Training Protocol

Hyper-parameters (identical for all 12 experiments):

- Epochs: 15 Batch size: 64
- Optimizer: Adam, learning rate = 10^{-3}
- Loss: Cross-Entropy
- Augmented data is **pre-generated** and stored in memory as tensors (no on-the-fly augmentation during training).
- Random seed: 42 (for reproducible subsampling)

5.6 Results

5.6.1 Accuracy Table

Generated/digit	350 real/digit	750 real/digit	1000 real/digit
0	96.35%	97.00%	96.90%
1000	96.80%	97.80%	98.10%
1500	96.45%	97.90%	98.10%
2000	97.20%	98.55%	97.80%

Table 9. Test accuracy (%) on 2,000 test images for each combination of real data budget

5.6.2 Learning Curves

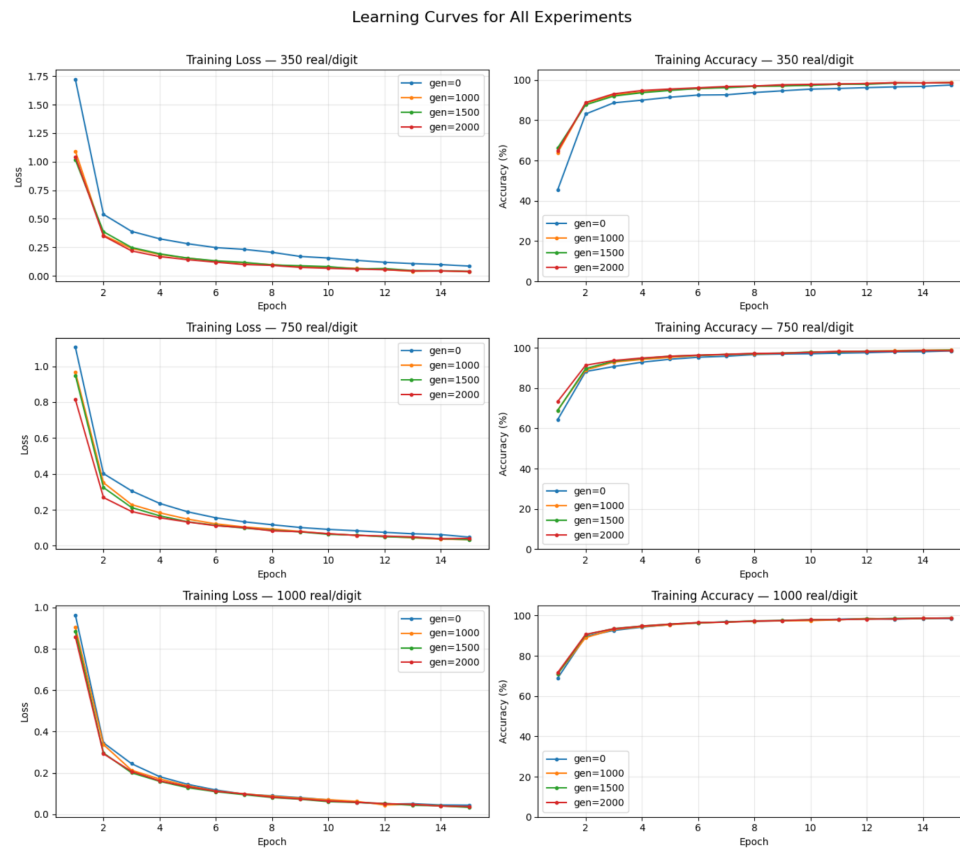


Figure 7. Training loss and accuracy over 15 epochs for all 12 experiments.

5.6.3 Bar Chart Comparison

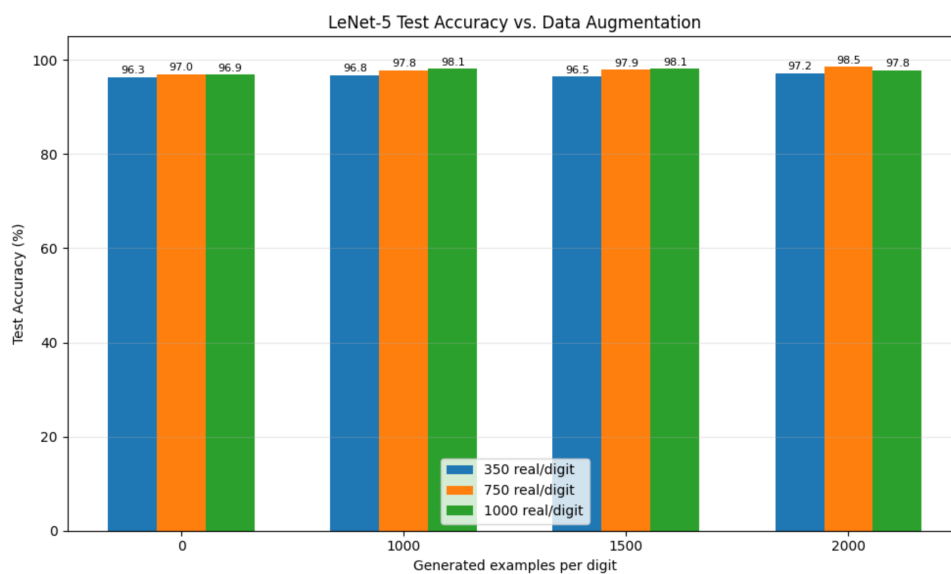


Figure 8. LeNet-5 test accuracy vs. data augmentation level, grouped by real data budget.

5.7 Analysis and Discussion

5.7.1 More Real Data Always Helps

Across all augmentation levels, increasing the real data budget from 350 to 750 to 1000 images per digit consistently improves test accuracy. Real data captures the *true distribution* of handwritten digits better than any transformation of a limited sample. This confirms that augmentation is a supplement, not a replacement, for real data collection.

5.7.2 Augmentation Boosts Low-Data Regimes Most

When real data is scarce (350 per digit), adding 1000 augmented examples per digit provides the largest absolute accuracy gain. The model benefits from seeing diverse geometric variations of the limited real samples, learning that the same digit can appear rotated, shifted, or noisy. With more real data (1000 per digit), the marginal benefit of augmentation is smaller because the network already has sufficient diversity in the original training set.

5.7.3 Diminishing Returns at High Augmentation Levels

Going from 1500 to 2000 generated examples per digit shows smaller improvements than going from 0 to 1000. Since all augmented images are derived from the *same* set of real base images via the same transformation pipeline, after a certain point the network has already learned the available variation. The effective diversity of the augmented set is bounded by the diversity of the source images and the transformation parameter ranges.

5.7.4 Augmentation as Free Labelling

Augmented images inherit their labels from the source image with no human verification needed. This makes data augmentation particularly valuable when labelling is expensive (e.g. medical imaging, rare-event detection). The only cost is compute time for generating and storing the augmented samples.

5.7.5 Regularisation Effect

Even when ample real data is available (1000 per digit), augmentation still improves generalisation. The random perturbations (rotation, translation, noise) act as an **implicit regulariser**: by seeing slightly different versions of each training sample every time, the network is discouraged from memorising exact pixel patterns and instead learns more robust, invariant features.

6 Problem 6: GAN-Based Synthetic Data Generation

6.1 Introduction

This problem explores using a **Generative Adversarial Network (GAN)** to synthesise new digit images and evaluates whether GAN-generated data can effectively augment the training set for a LeNet-5 classifier. The problem is divided into three sub-parts:

1. **Problem 6.1:** Train a conditional DCGAN (cDCGAN) on 350 real images per digit and generate sample images. Evaluate visual quality and report training time.
2. **Problem 6.2:** Use the trained GAN to generate synthetic data at various scales (0, 1000, 1500, 2000 per digit) and train a LeNet-5 classifier on combined real + synthetic data. Compare across three real data budgets (350, 750, 1000 per digit).
3. **Problem 6.3:** Combine traditional augmentation (from Problem 5) with GAN-generated synthetic data. Compare the best combination (using only 350 real images) against the baseline of 1000 real images per digit.

References:

- DCGAN paper: Radford et al., “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” arXiv:1511.06434.
- Conditional GAN (cGAN): Mirza & Osindero, “Conditional Generative Adversarial Nets,” arXiv:1411.1784.

System: Linux (WSL2), x86_64, NVIDIA RTX A1000 6GB Laptop GPU, CUDA 11.8, PyTorch 2.7.1+cu118.

6.2 Dataset

The same ReducedMNIST dataset is used. For GAN training, a subset of **350 real images per digit** (3,500 total) is selected with a fixed random seed. Images are resized to 32×32 and normalised to $[-1, 1]$ (matching the Tanh output range of the generator).

6.3 GAN Architecture: Conditional DCGAN

The cDCGAN consists of a **Generator** (1,227,381 parameters) that maps a latent noise vector $\mathbf{z}$ and a class label to a 32×32 greyscale image, and a **Discriminator** (714,997 parameters) that classifies (image, label) pairs as real or fake.

6.3.1 Generator

Component	Operation	Output Shape
Label Embedding	Embedding($10 \rightarrow 50$) + Linear($50 \rightarrow 64$)	$(B, 1, 8, 8)$
Latent Projection	Linear($100 \rightarrow 128 \times 8 \times 8$) + ReLU	$(B, 128, 8, 8)$
Concatenation	Cat along channel dim	$(B, 129, 8, 8)$
ConvTranspose1	ConvT2d($129 \rightarrow 128, 4 \times 4, \text{stride}=2, \text{pad}=1$) + BN + ReLU	$(B, 128, 16, 16)$
ConvTranspose2	ConvT2d($128 \rightarrow 64, 4 \times 4, \text{stride}=2, \text{pad}=1$) + BN + ReLU	$(B, 64, 32, 32)$
Output Conv	Conv2d($64 \rightarrow 1, 3 \times 3, \text{pad}=1$) + Tanh	$(B, 1, 32, 32)$

Table 10. Generator architecture. B = batch size. Output range: $[-1, 1]$.

6.3.2 Discriminator

Component	Operation	Output Shape
Label Embedding	Embedding($10 \rightarrow 50$) + Linear($50 \rightarrow 1024$)	$(B, 1, 32, 32)$
Input Concat	Cat(image, label_map)	$(B, 2, 32, 32)$
Conv1	Conv2d($2 \rightarrow 64, 4 \times 4, \text{stride}=2, \text{pad}=1$) + LeakyReLU(0.2)	$(B, 64, 16, 16)$
Conv2	Conv2d($64 \rightarrow 128, 4 \times 4, \text{stride}=2, \text{pad}=1$) + BN + LeakyReLU(0.2)	$(B, 128, 8, 8)$
Conv3	Conv2d($128 \rightarrow 256, 4 \times 4, \text{stride}=2, \text{pad}=1$) + BN + LeakyReLU(0.2)	$(B, 256, 4, 4)$
Classifier	Flatten + Linear($256 \times 4 \times 4 \rightarrow 1$) + Sigmoid	$(B, 1)$

Table 11. Discriminator architecture.

6.3.3 Design Choices (Following DCGAN Paper)

- **Batch Normalisation** in both G and D for training stability.
- **ReLU** activations in the Generator; **LeakyReLU** ($\alpha = 0.2$) in the Discriminator.
- **Transposed convolutions** (fractionally-strided) for spatial upsampling in the Generator — avoids checkerboard artefacts better than naive upsampling.
- **Conditional label embedding** enables class-specific generation: the Generator receives the target digit as input, and the Discriminator verifies that the image matches the claimed label.
- **Weight initialisation:** $\mathcal{N}(0, 0.02)$ for convolutional weights; $\mathcal{N}(1, 0.02)$ for BatchNorm weights.

6.4 GAN Training Protocol

Hyper-parameters:

- Epochs: 100 Batch size: 64
- Optimizer: Adam for both G and D, learning rate = 2×10^{-4} , $\beta_1 = 0.5$, $\beta_2 = 0.999$
- Loss: Binary Cross-Entropy (BCE)
- Latent dimension: $z \in \mathbb{R}^{100}$, sampled from $\mathcal{N}(0, I)$
- Training data: 3,500 images (350 per digit), normalised to $[-1, 1]$

Training procedure:

1. **Discriminator step:** feed real images with label “real” (target = 1) and generated images with label “fake” (target = 0). Compute BCE loss and update D.
2. **Generator step:** generate fake images and pass through D with target = 1 (fool the discriminator). Compute BCE loss and update G.

6.5 Problem 6.1: Generated Examples and Quality Assessment

6.5.1 GAN Loss

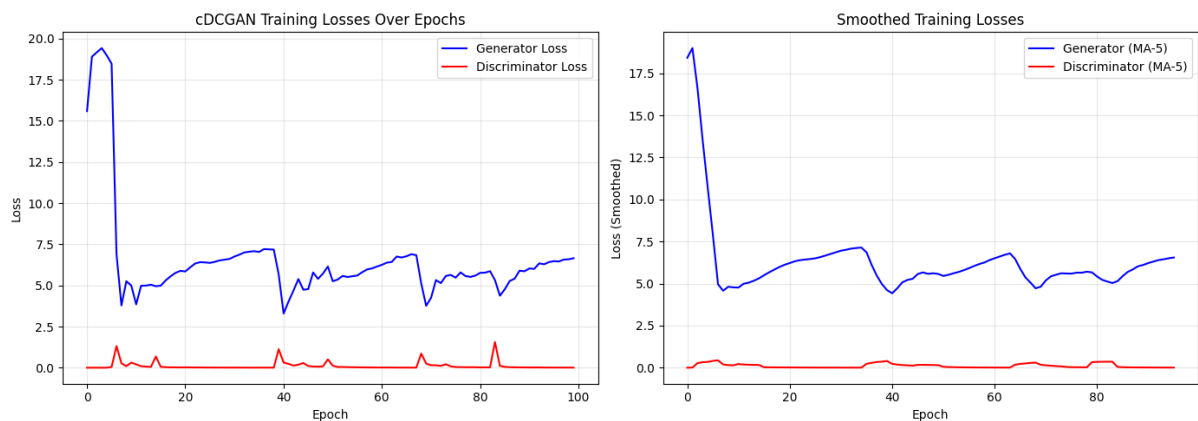


Figure 9. cDCGAN training losses over 100 epochs.

6.5.2 Generated Samples: Real vs. Fake Comparison

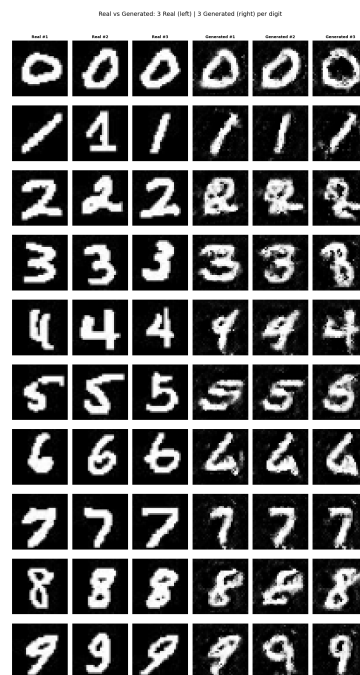


Figure 10. Real training images (left 3 columns) vs. GAN-generated images (right 3 columns)

6.5.3 Comments on Generated Quality

Visual quality:

- Generated digits are **recognisable** and capture the overall shape and stroke patterns of each digit class.
- Since the GAN was trained on only 350 examples per digit, some artefacts are visible: slight blurriness, occasional inconsistencies in stroke thickness, and less sharp edges compared to real images.
- Digits with simpler structure (0, 1, 7) tend to look more realistic than complex digits (4, 8, 9) which have more varied handwriting styles.

Timing:

- GAN training (100 epochs on 3,500 images): **115.94 s** (1.93 min).
- Generation of 30 images (3 per digit): **0.21 s**.
- Total time (train + generate): **116.15 s**.
- The bulk of compute cost is in *training* the GAN, not in generating new samples — once trained, the generator can produce unlimited samples at negligible cost.

6.6 Problem 6.2: Synthetic Data Scaling Experiments

6.6.1 Experiment Setup

The trained cDCGAN is used to generate synthetic data at three scales (1000, 1500, 2000 per digit). These are combined with real data subsets of different sizes. A **LeNet-5 classifier** (same architecture as Problem 5) is trained for each combination and evaluated on the held-out test set.

Synthetic/digit	350 real	750 real	1000 real
0	350 real only	750 real only	1000 real only
1000	350 real + 1000 syn	750 real + 1000 syn	1000 real + 1000 syn
1500	350 real + 1500 syn	750 real + 1500 syn	1000 real + 1500 syn
2000	350 real + 2000 syn	750 real + 2000 syn	1000 real + 2000 syn

Table 12. Experiment grid: combinations of real and GAN-generated synthetic data.

Classifier training: Same protocol as Problem 5 (Adam, $\text{lr}=10^{-3}$, 15 epochs, batch size 64, Cross-Entropy loss).

Synthetic data generation: 20,000 images (2000 per digit) pre-generated in **0.86 s**.

6.6.2 Results Table

Synthetic/digit	350 real/digit	750 real/digit	1000 real/digit
0	95.55%	96.45%	97.25%
1000	95.80%	97.75%	97.90%
1500	96.60%	96.40%	97.60%
2000	95.60%	96.95%	96.85%

Table 13. Test accuracy for each combination of real data and GAN-generated synthetic data.

6.6.3 Bar Chart

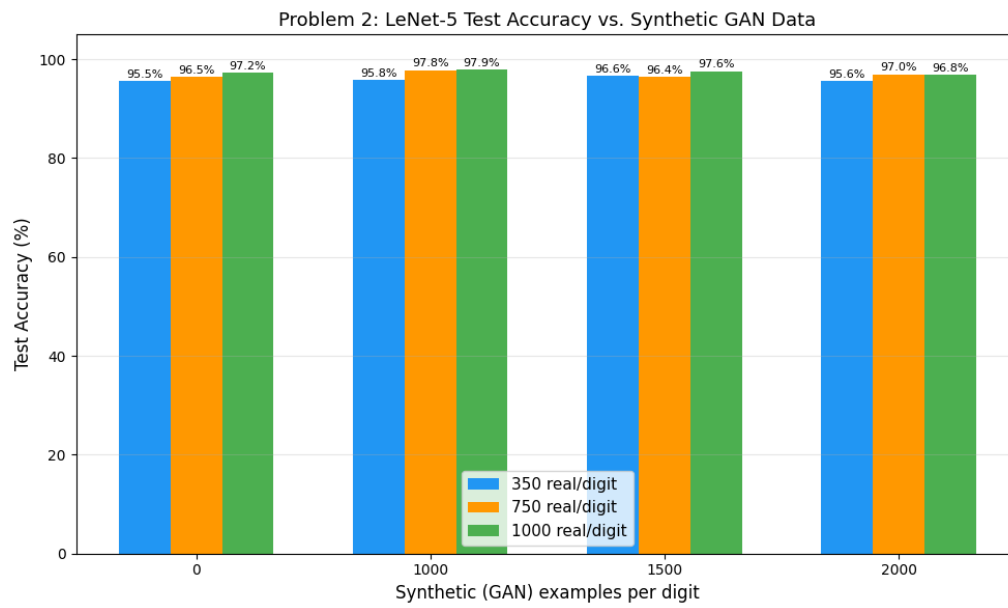


Figure 11. LeNet-5 test accuracy vs. GAN-generated synthetic data.

6.6.4 Learning Curves

Problem 2: Learning Curves (Training Loss & Accuracy)

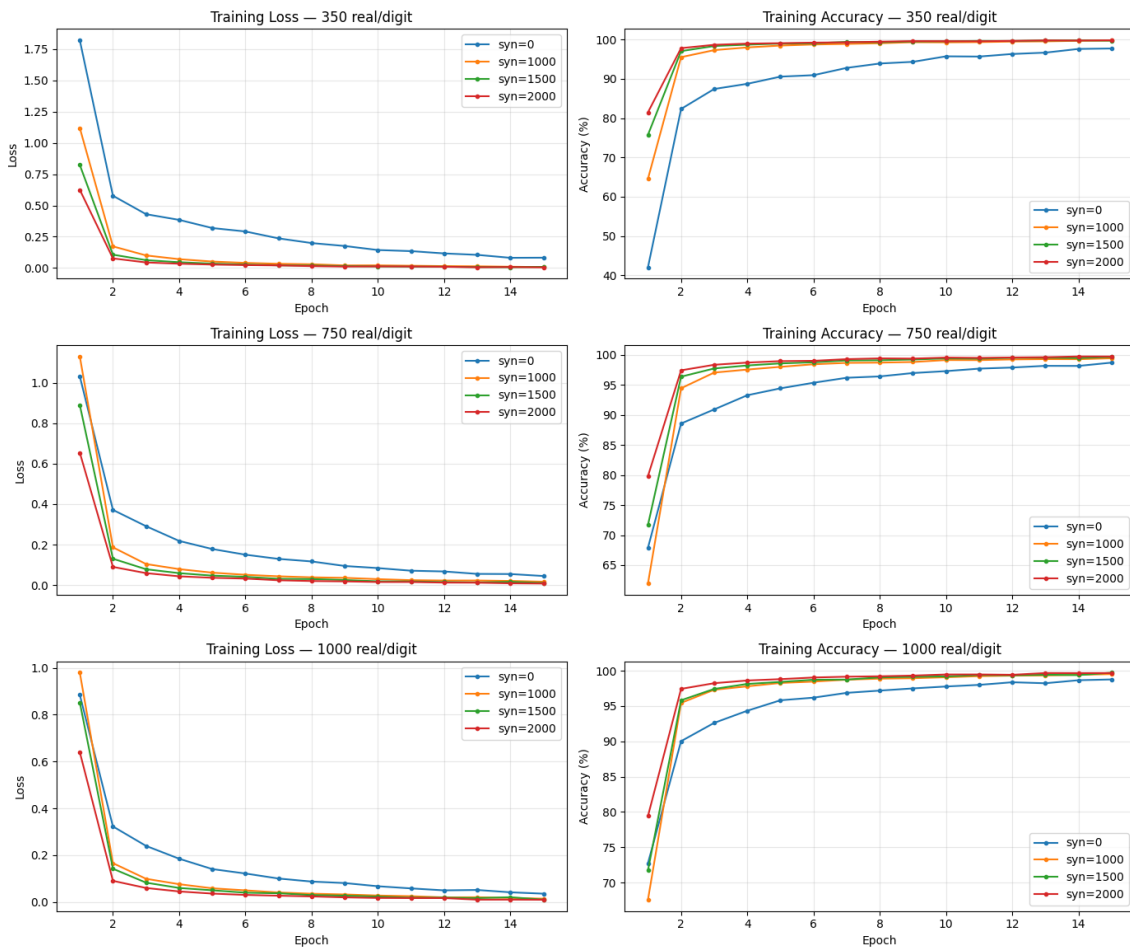


Figure 12. Training loss and accuracy over 15 epochs for all 12 synthetic data experiments.

6.6.5 Accuracy

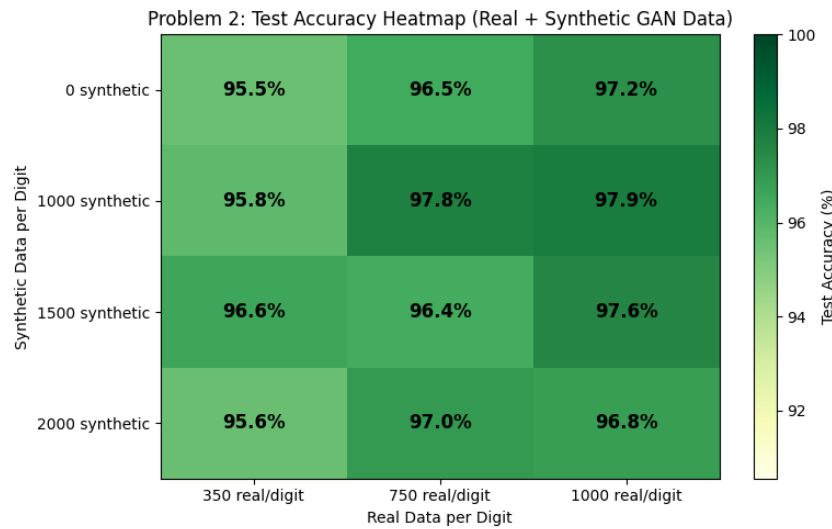


Figure 13. Test accuracy heatmap

6.6.6 Analysis

Synthetic data helps most in low-data regimes: Adding GAN-generated synthetic data to 350 real images per digit provides a modest improvement (95.55% $\rightarrow$ 96.60% at 1500 syn/digit). The GAN provides structural diversity — new stroke patterns, thickness variations, and digit shapes — that the classifier has not seen in the limited real set. As the real data budget increases (750, 1000), the marginal benefit of synthetic data diminishes because the real set already covers most of the variation in the true distribution.

Diminishing returns with more synthetic data: Going from 1000 to 2000 synthetic images per digit does not consistently improve accuracy — in fact accuracy sometimes decreases (e.g. 750 real: 97.75% at 1000 syn $\rightarrow$ 96.95% at 2000 syn). All synthetic images are produced by the same GAN trained on only 350 real examples, so the generator’s output diversity is bounded. Excessive synthetic data can even dilute the signal from real examples.

Real data remains superior: Across all synthetic counts, increasing real data (350 $\rightarrow$ 750 $\rightarrow$ 1000 per digit) consistently outperforms adding more synthetic data. The best result (**97.90%**) is achieved with 1000 real + 1000 synthetic per digit.

GAN quality bottleneck: The GAN was trained on only 350 images per digit. This limits the diversity and quality of generated samples. A GAN trained on more data would produce sharper, more varied samples and better represent the tails of the digit distribution.

6.7 Problem 6.3: Combining Augmented + Synthetic Data

6.7.1 Motivation

Traditional augmentation (Problem 5) and GAN synthesis (Problem 6.2) provide **complementary** forms of diversity:

- **Augmentation** applies geometric transformations and noise to existing images, preserving exact structural details while varying pose and appearance.
- **GAN synthesis** generates entirely new images that may capture novel stroke patterns not present in the real set.

Combining both should yield a training set that is both geometrically diverse (from augmentation) and structurally diverse (from the GAN), providing the strongest regularisation effect.

6.7.2 Experiment Setup

All combination experiments use **350 real images per digit** as the base. Various mixtures of augmented and synthetic data are added. The target is to match or exceed the performance of the **1000 real/digit baseline** (97.30%).

Configuration	Aug/digit	Syn/digit	Total/digit
350 real only	0	0	350
350 real + 1000 augmented	1000	0	1350
350 real + 1000 synthetic	0	1000	1350
350 real + 500 aug + 500 syn	500	500	1350
350 real + 1000 aug + 1000 syn	1000	1000	2350
350 real + 1500 aug + 500 syn	1500	500	2350
350 real + 500 aug + 1500 syn	500	1500	2350
350 real + 1000 aug + 1500 syn	1000	1500	2850
350 real + 2000 augmented	2000	0	2350
350 real + 2000 synthetic	0	2000	2350
350 real + 1000 aug + 2000 syn	1000	2000	3350
Baseline: 1000 real/digit	0	0	1000

Table 14. Combination experiment configurations.

6.7.3 Results

Configuration	Test Acc. (%)
350 real only	94.90
350 real + 1000 augmented	97.75
350 real + 1000 synthetic	96.50
350 real + 500 aug + 500 syn	97.00
350 real + 1000 aug + 1000 syn	97.75
350 real + 1500 aug + 500 syn	97.20
350 real + 500 aug + 1500 syn	97.05
350 real + 1000 aug + 1500 syn	96.95
350 real + 2000 augmented	97.40
350 real + 2000 synthetic	96.15
350 real + 1000 aug + 2000 syn	97.55
Baseline: 1000 real/digit	97.30

Table 15. Test accuracy for each combination.

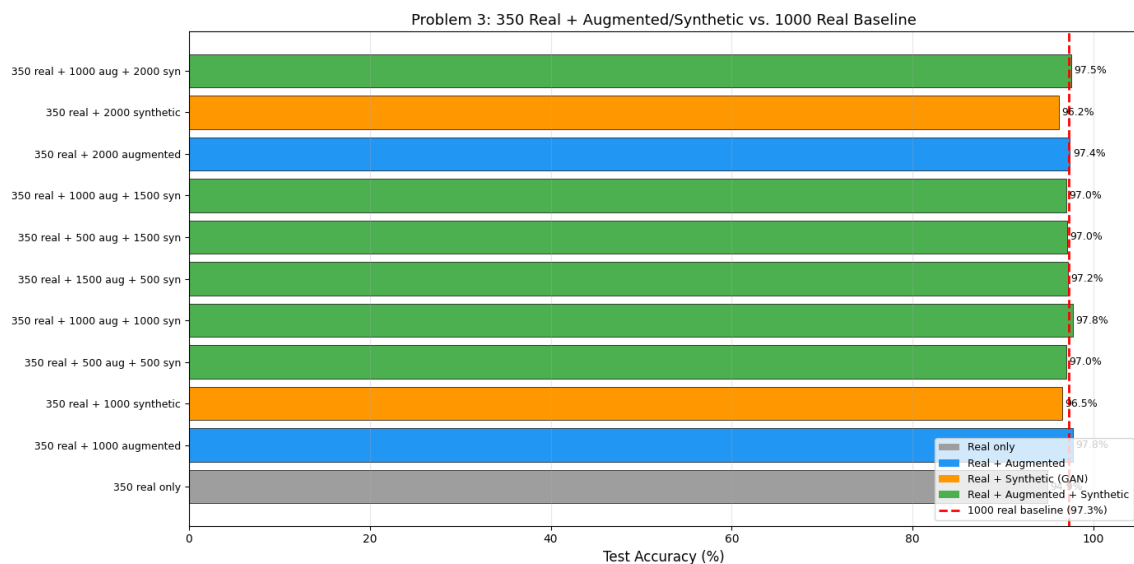


Figure 14. Test accuracy for 350-real combinations vs. the 1000-real baseline

6.7.4 Analysis and Conclusions

Best combination matches the 1000-real baseline: The configurations “350 real + 1000 augmented” and “350 real + 1000 aug + 1000 syn” both achieve **97.75%**, exceeding the 1000-real baseline of 97.30%. This means we can reduce real data requirements by **65%** (from 1000 to 350 per digit) with no loss in performance.

Augmentation outperforms synthetic data alone: With the same budget (1000 generated/digit), augmentation alone (97.75%) outperforms synthetic GAN data alone (96.50%). This suggests that for this dataset, geometric diversity from transformations is more valuable than the structural diversity from a GAN trained on limited data.

Combined data provides robustness: While the best augmented-only and combined configurations tie at 97.75%, the combined approach is more robust across different mix ratios — multiple combined configurations (97.00–97.75%) exceed the synthetic-only results (96.15–96.50%).

Diminishing returns from excess synthetic data: Adding more than 1000 synthetic images per digit provides no benefit and can even slightly reduce accuracy. The GAN’s output diversity is bounded by its training on only 350 real images.

Summary:

- 350 real only: 94.90%
- Best augmented only (1000 aug): 97.75%
- Best synthetic only (1500 syn): 96.60%
- Best combined (1000 aug + 1000 syn): **97.75%**
- Baseline (1000 real): 97.30%

6.8 Comparison: Traditional Augmentation (Problem 5) vs. GAN Synthesis (Problem 6)

Criterion	Augmentation (Prob. 5)	GAN Synthesis (Prob. 6)
Training required	No	Yes (100 epochs, 115.94 s)
Generation speed	Fast (transforms)	Fast (0.86 s for 20k images)
Diversity type	Geometric/noise	Structural/stylistic
Label correctness	Guaranteed	Guaranteed (conditional)
Compute cost	Negligible	Moderate (GPU minutes)
Quality at low data	Good	Limited by GAN training set
Best use case	Always applicable	When structural diversity needed

Table 16. Comparison of traditional augmentation vs. GAN-based synthesis.

6.8.1 Final Recommendation

For the ReducedMNIST task with limited labelled data:

1. **Always apply traditional augmentation** — it is free, effective, and universally applicable. It alone can match the 1000-real baseline.
2. **Add GAN synthetic data** when compute resources are available — the complementary diversity provides additional robustness.
3. **Prioritise collecting more real data** when possible — no generation method fully replaces the true data distribution.

Timing summary:

- GAN training: 115.94 s (1.93 min)
- Synthetic generation (20,000 images): 0.86 s
- Total GAN pipeline: 122.33 s